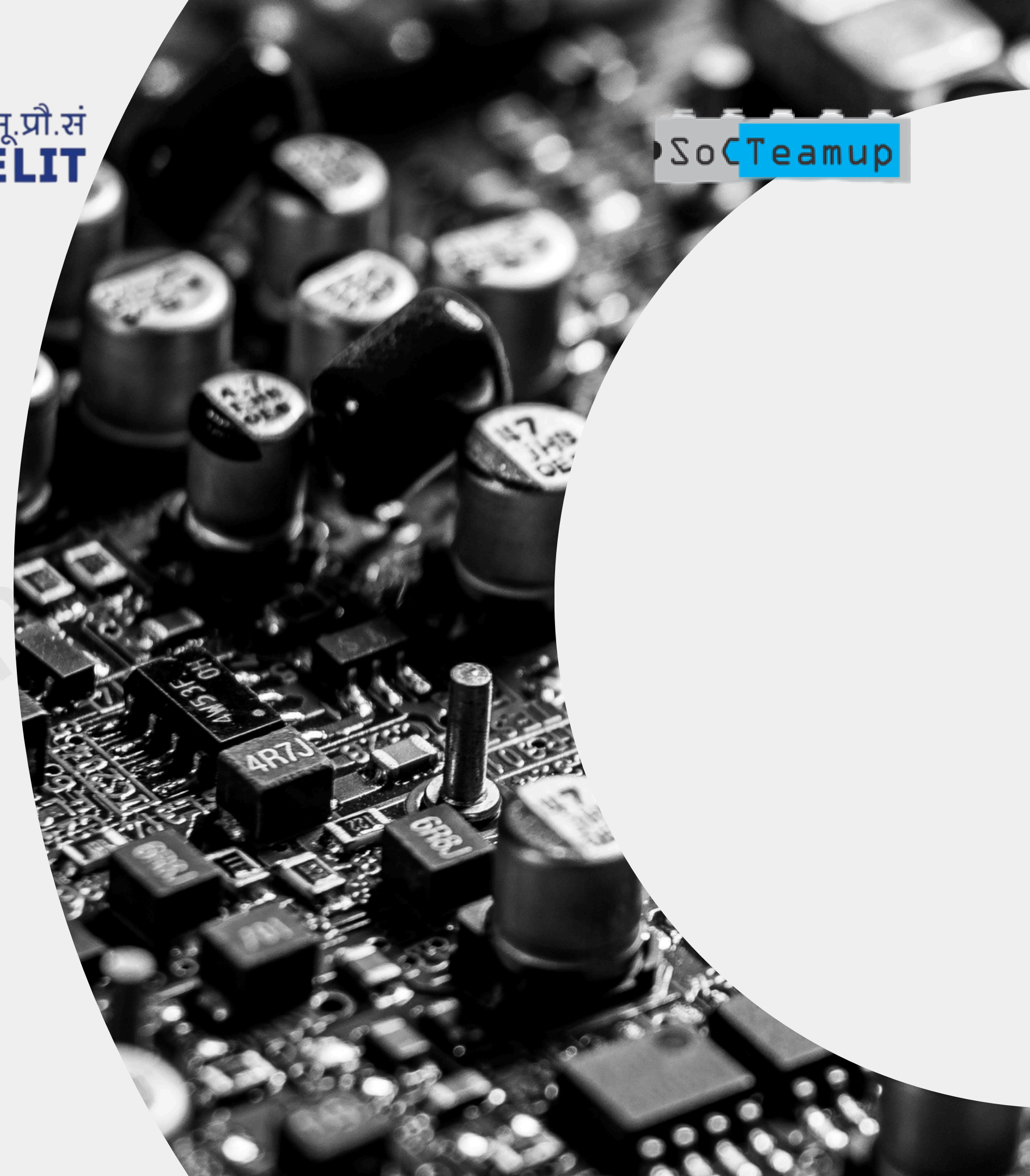


CDC/RDC Techniques And Synchronization Methodologies





Outline

What is a Synchronizer?
CDC Design Goals and Hazards
2-FF Synchronizer for Control Signals?
RTL Code for 2-FF Synchronizer
Pulse Synchronization Techniques
Multi-bit Data Synchronization – Why Not 2FF?
Gray Code for Status Transfers
Handshake-Based Synchronization
Reset Domain Crossing (RDC)
2-FF Reset Synchronizer
Glitch-Free Reset Multiplexing
CDC/RDC Design Checklist
Tools for CDC/RDC Detection

➔ What is a Synchronizer?

A synchronizer is a small logic circuit that safely transfers a signal between asynchronous domains.

Why use it?	Used in:
• Avoids metastability • Aligns timing with destination clock • Ensures predictable behavior	• Clock Domain Crossing (CDC) • Reset Domain Crossing (RDC)

➔ CDC Design Goals and Hazards

CDC = Clock Domain Crossing → source and destination clocks are not aligned

Design Goals:	Strategy:
• Avoid metastability • Avoid partial or corrupted data • Avoid false triggering	• Identify CDC paths early • Choose correct synchronization based on signal type (control/data)

➔ 2-FF Synchronizer – Control Signals

- It is used for 1-bit control signals (e.g., enable, valid, flag)

Structure:	Tip:
• 2 Flip-Flops in destination domain • First FF captures async input • Second FF stabilizes it	• Keep both FFs close to reduce skew and routing delay

RTL Code for 2-FF Synchronizer

Verilog code:

```
always @(posedge clk_dest) begin
    sync1 <= async_flag;
    sync2 <= sync1;
end
assign sync_flag = sync2;
```

Key Notes

- 'async_flag' from source domain
- 'sync_flag' is safe in destination domain
- Best for simple CDC control logic

Pulse Synchronization Technique

- **Problem**
 - A 1-clock-cycle pulse in source domain may be missed in destination
- **Solution**



**CONVERT PULSE TO LEVEL +
USE SYNCHRONIZER**



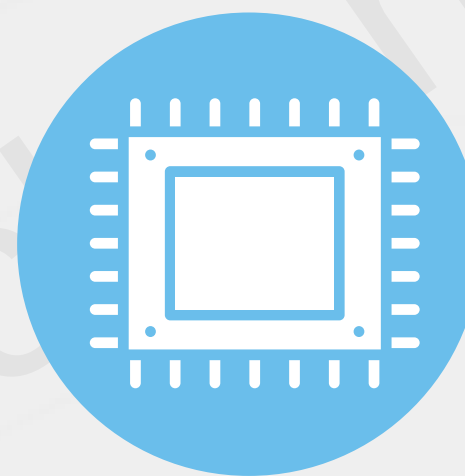
**STRETCH PULSE
(TOGGLE LOGIC)**

Pulse Synchronization Technique

- **Problem**
 - A 1-clock-cycle pulse in source domain may be missed in destination
- **Solution**



**USE HANDSHAKE TO
CONFIRM RECEPTION**



**USED IN:
INTERRUPTS,
COMMAND TRIGGERS,
ONE-SHOT FLAGS**

➔ Multi-bit Data Synchronization

– Why Not 2FF?

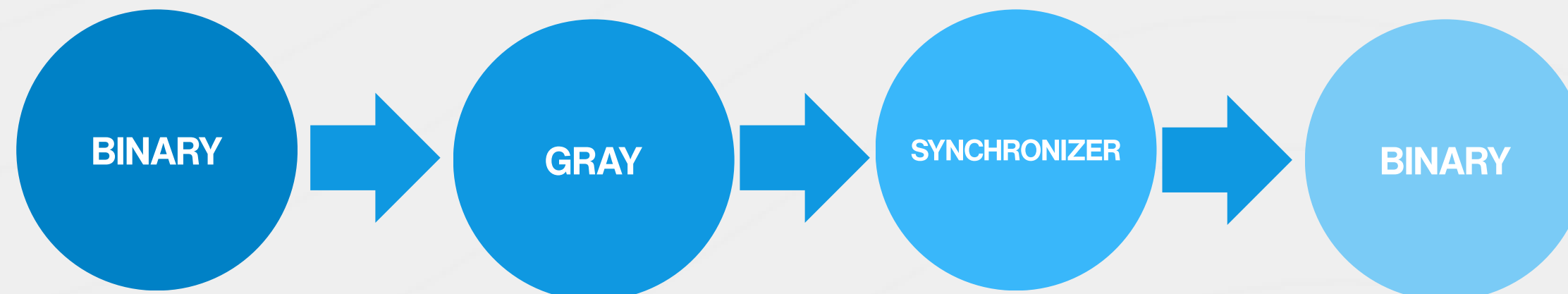
- Don't use 2FF for buses!

Why?	Correct Approaches:
• Each bit stabilizes differently • Can lead to corrupted data at destination	• Use async FIFO for data • Use handshake protocol • Use Gray code for status

➔ Gray Code for Status Transfers

- Gray Code = only 1 bit changes per transition

Why Use?	Applications:
• Reduces glitching during synchronization • Ensures stable state transitions	• FIFO read/write pointers • Token counters



Handshake-Based Synchronization

- Reliable protocol for request-response CDC

Handshake Steps:	Integration Files:
• Source raises req • Destination latches and sets ack • Source drops req after seeing ack	• Config register updates • Safe message passing • FSM triggers

➔ Reset Domain Crossing (RDC) Explained

- RDC = Reset signal affects logic in another clock domain

Dangers:	Integration Files:
• Reset glitches can cause invalid states • Async de-assertion leads to metastability	• Assert reset asynchronously • De-assert synchronously using 2-FF reset sync

2-FF Reset Synchronizer

Example:

```
always @(posedge clk or negedge rst_n_async)
    if (!rst_n_async) begin
        sync1 <= 1'b0;
        sync2 <= 1'b0;
    end else begin
        sync1 <= 1'b1;
        sync2 <= sync1;
    end
```

```
assign rst_n_sync = sync2;
```

- **Purpose:**

- Ensures glitch-free reset release in local clock domain
- Robust for IPs receiving resets from external logic

➔ Glitch-Free Reset Multiplexing

- RDC = Reset signal affects logic in another clock domain

Challenge:	Wrong Method:	Correct Method:
• Multiple reset sources (e.g., power-on, debug, watchdog)	• Using asynchronous MUX → introduces glitches	• Use FSM or control logic to switch resets • Use registered reset release paths

- Always verify with testbenches simulating overlap

➔ CDC/RDC Design Checklist

CDC:	Use 2FF for control signals Async FIFO or handshake for multi-bit data Don't use 2FF per bit on buses
RDC:	Assert reset async, de-assert sync Use 2FF reset synchronizer Avoid combinational logic in reset path
General:	Label crossings in schematics Simulate transition edge cases

Tools for CDC/RDC Detection

- Simulation isn't enough — use CDC/RDC checkers

Commercial Tools:	Open Source:	Use tools:
• SpyGlass CDC • Questa CDC • VC Formal CDC	• SymbiYosys (assertion-based) • Verilator (debug & waveform)	• After RTL integration • Before synthesis and tapeout

Thank You

echiphu

